

Architettura degli Elaboratori

Cache set-associative e multi-livello



SAPIENZA
UNIVERSITÀ DI ROMA

Davide Polese

davide.polese@uniroma1.it

Special thanks and credits:

Alessandro Checco, Andrea Sterbini, Iacopo

Masi, Claudio di Ciccio

[S&PdC]

5.1, 5.3 – 5.4



Argomento

Problema:

la RAM è **molto più lenta** del processore
(circa 10-100 volte più lenta)

Una memoria più veloce è **molto più costosa**

1. Principio di località temporale

- «un programma tende ad accedere allo stesso elemento in momenti vicini tra loro»

2. Principio di località spaziale

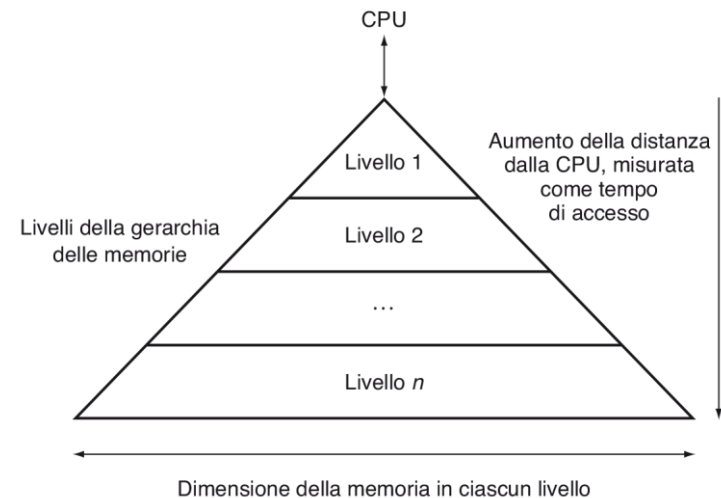
- «un programma tende ad accedere successivamente elementi di memoria vicini tra loro»

• Idea 1:

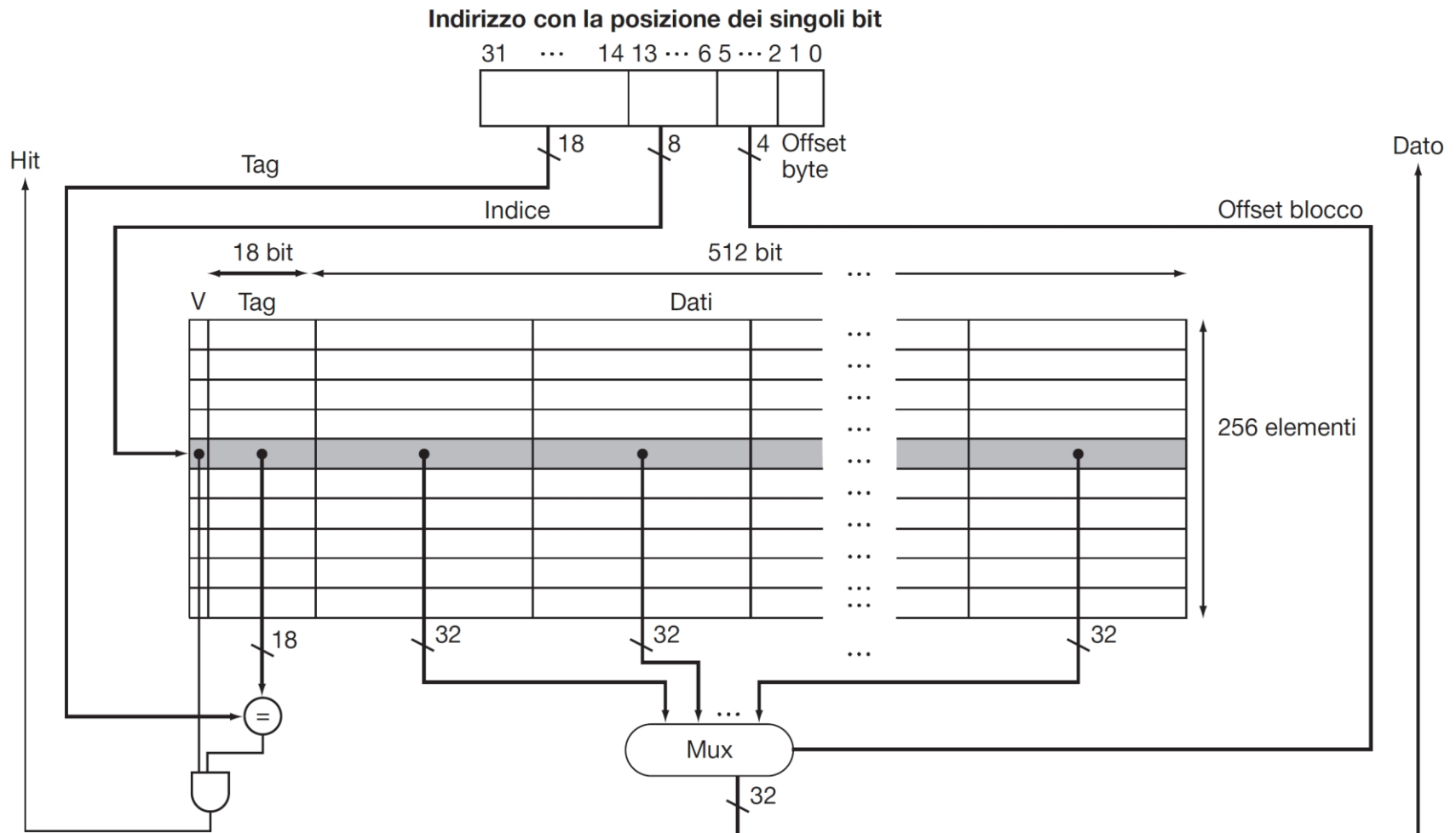
- mettiamo da parte in una **piccola** memoria **veloce (cache)** solo i **dati più usati**

• Idea 2:

- quando memorizziamo un dato **mettiamo da parte anche i dati vicini**



Cache a mappatura diretta



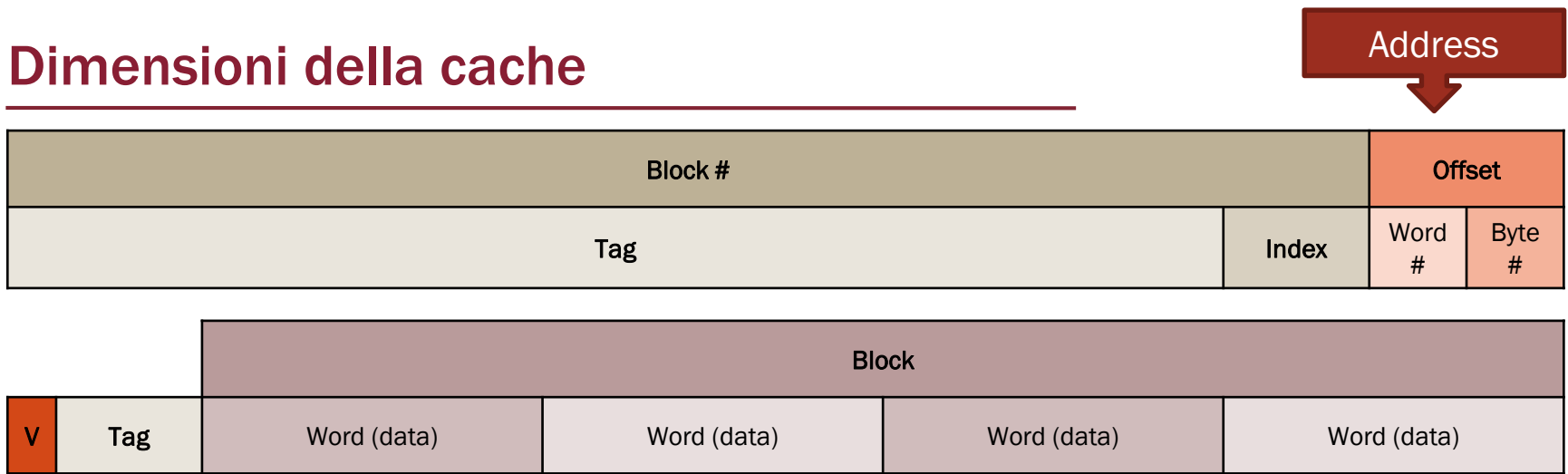
RAM

Table 2: SDRAM Devices Comparison

Items	DDR3 SDRAM	DDR2 SDRAM	DDR SDRAM
Clock frequency	400/533/667/ 800 MHz	200/266/333/400 MHz	100/133/166/200 MHz
Transfer data rate	800/1066/1333/ 1600 Mbps	400/533/667/800 Mbps	200/266/333/400 Mbps
I/O width	x4/x8/x16	x4/x8/x16	x4/x8/x16/x32
Prefetch bit width	8-bit	4-bit	2-bit
Clock input	Differential clock	Differential clock	Differential clock
Burst length	8, 4 (Burst chop)	4, 8	2, 4, 8
Data strobe	Differential data strobe	Differential data strobe	Single data strobe
Supply voltage	1.5V	1.8V	2.5V
Interface	SSTL_15	SSTL_18	SSTL_2
/CAS latency (CL)	5, 6, 7, 8, 9, 10 clock	3, 4, 5 clock	2, 2.5, 3 clock
On die termination (ODT)	Supported	Supported	Unsupported
Component package	FBGA	FBGA	TSOP(II) / FBGA / LQFP

Latenza della RAM: i dati in ram sono disponibili sul bus dati dopo diversi cicli di clock.

Dimensioni della cache



Dati:

- Cache (direct mapping) con 2^n blocchi
- Blocchi di dimensione 2^m word da 4 byte ciascuna, cioè 2^{m+2} byte (2^{m+5} bit)
- 1 bit di validità

Dimensione del campo tag:

$$32 - n - m - 2 \quad \text{V}$$

Dimensione della cache in bit:

$$2^n \times [2^m \times 32 + (32 - n - m - 2) + 1]$$

Esempio: cache con:

$$\# \quad \underbrace{\quad \quad \quad}_{\text{blocco}} \quad \underbrace{\quad \quad \quad}_{\text{tag}}$$

- blocchi da 16 word $\rightarrow m = 4$

- 256 linee $\rightarrow n = 8$

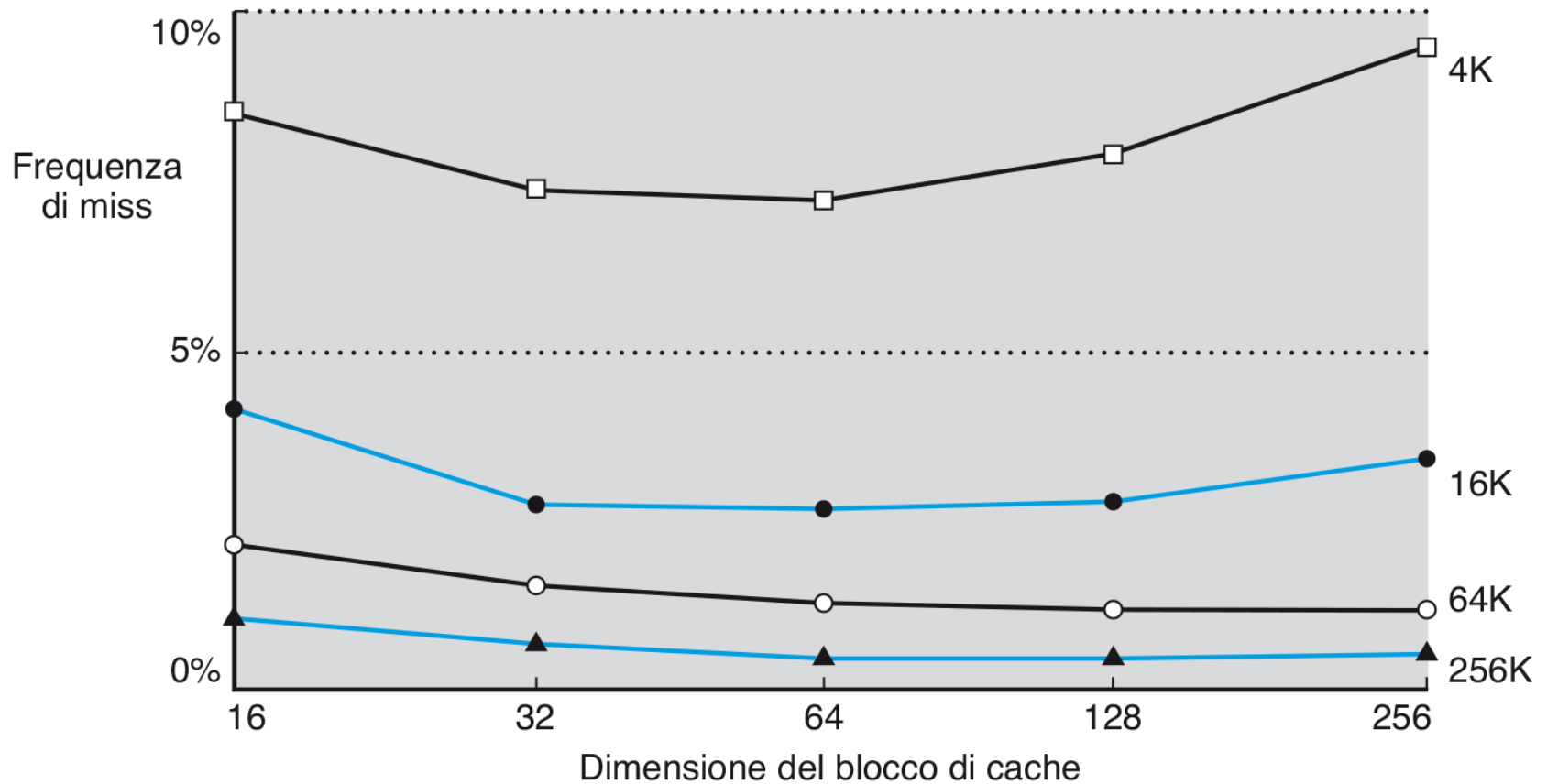
Dimensione del campo Tag:

$$32 - 8 - 4 - 2 = 18 \text{ bit}$$

Dimensione della cache in bit:

$$256 \times [16 \times 32 + 18 + 1] = 2^8 \times 531 = 132,75 \text{ Kibibit}$$

La dimensione del blocco non è la panacea



Altri tipi di mapping

Come migliorare la percentuale di HIT?

- Direct mapping

- ogni blocco viene salvato in una linea fissata (che dipende dal numero di blocco)

pro: hardware più semplice e meno costoso

è facile determinare in quale linea cercare il dato

$$\#linea = \#blocco \% N$$

contro: blocchi diversi sono mappati sulla stessa linea →

se gli accessi a quei blocchi si alternano si verificano molti miss

(trashing)

- Fully-associative mapping

- un blocco può essere posto in una linea qualsiasi

pro: massima flessibilità

contro: hardware più complesso e più costoso

- Set-associative mapping

- le linee sono organizzate in **S** insiemi (set)
- ogni blocco è disposto in una qualsiasi delle linee del set fissato (che dipende dal #blocco)

$$\#set = \#blocco \% S$$

Attenzione: da qui in poi
adottiamo la
nomenclatura per il set-
associative mapping usata
in G.R. Wilson, in
Embedded Systems and
Computer Architecture,
2002

Associatività crescente

Cache con **8 blocchi**
organizzati mediante
diversi gradi di
associatività

$\#v\text{ie} = \#l\text{inee per set}$

scelta del set è
prefissata

$\#set = \#blocco \% S$

se ho un solo set un
blocco può essere posto
in una linea qualsiasi

**Cache set-associativa a una via
(a mappatura diretta)**

Linea	Tag	Dato
0		
1		
2		
3		
4		
5		
6		
7		

Cache set-associativa a due vie

Set	Tag	Dato	Tag	Dato
0				
1				
2				
3				

Cache set-associativa a quattro vie

Set	Tag	Dato	Tag	Dato	Tag	Dato	Tag	Dato
0								
1								

Cache set-associativa a otto vie (completamente associativa)

Tag	Dato	Tag	Dato	Tag	Dato	Tag	Dato	Tag	Dato	Tag	Dato	Tag	Dato	Tag	Dato

Associatività crescente – come si riempie [0 1 2 3 4 8]?

Cache con **8 blocchi**
organizzati mediante
diversi gradi di
associatività

#vie = #linee per set

scelta del set è
prefissata

#set = #blocco % S

se ho un solo set un
blocco può essere posto
in una linea qualsiasi

**Cache set-associativa a una via
(a mappatura diretta)**

Linea	Tag	Dato
0		
1		
2		
3		
4		
5		
6		
7		

Cache set-associativa a due vie

Set	Tag	Dato	Tag	Dato
0				
1				
2				
3				

Cache set-associativa a quattro vie

Set	Tag	Dato	Tag	Dato	Tag	Dato	Tag	Dato
0								
1								

Cache set-associativa a otto vie (completamente associativa)

Tag	Dato	Tag	Dato	Tag	Dato	Tag	Dato	Tag	Dato	Tag	Dato	Tag	Dato	Tag	Dato

Associatività crescente

Cache con **8 blocchi** organizzati mediante diversi gradi di associatività.

#set × #vie

Cache set-associativa a una via
(a mappatura diretta)

Linea	Tag	Dato
0		
1		
2		
3		
4		
5		
6		
7		

8×1

Cache set-associativa a due vie

Set	Tag	Dato	Tag	Dato
0				
1				
2				
3				

4×2

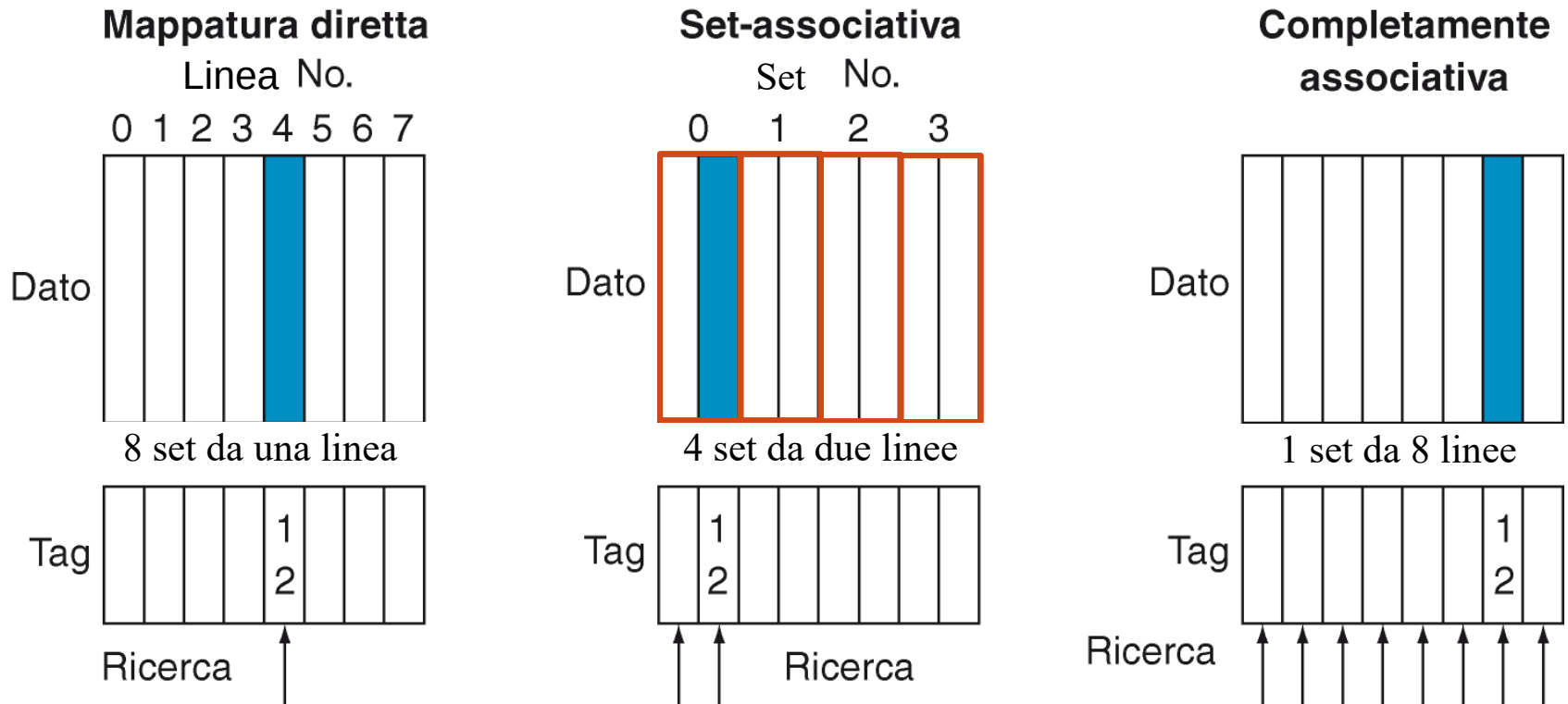
Cache set-associativa a quattro vie

Set	Tag	Dato	Tag	Dato	Tag	Dato	Tag	Dato
0								
1								

2×4

Associatività crescente

Livelli di associatività

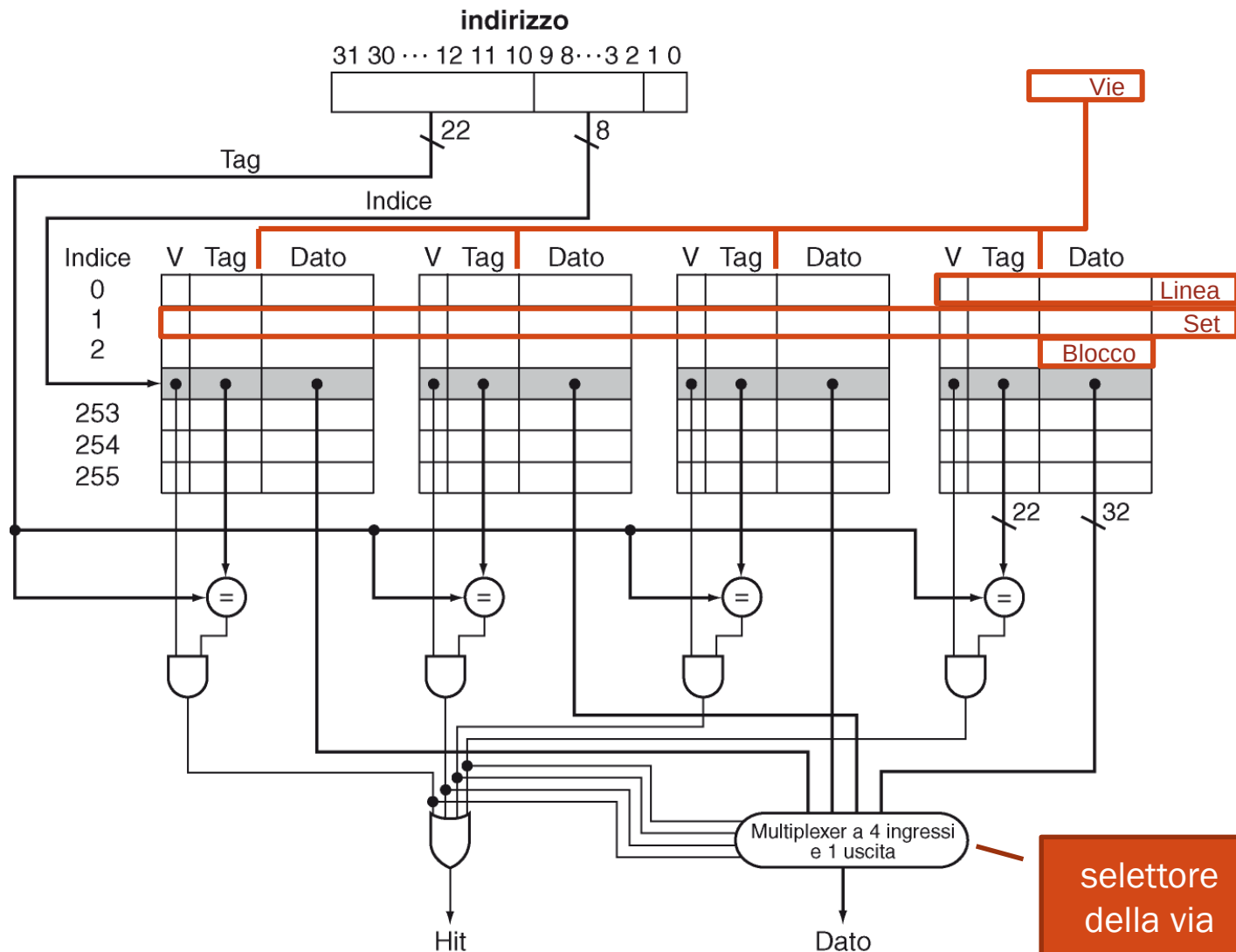


Associatività	Frequenza di miss
1	10,3%
2	8,6%
4	8,3%
8	8,1%

Cache set-associativa a 4 vie con blocco da 1 word

È necessario
un comparatore
per ciascuna via

Fully-associative
→ tanti
comparatori
quante sono
le vie (invece di
4 comparatori
me ne servono
1024)



Quanto è grande una cache? (update)

Per ogni **via** abbiamo una «tabella»

ogni tabella contiene **S set** con blocchi di **N word**

ogni linea contiene:

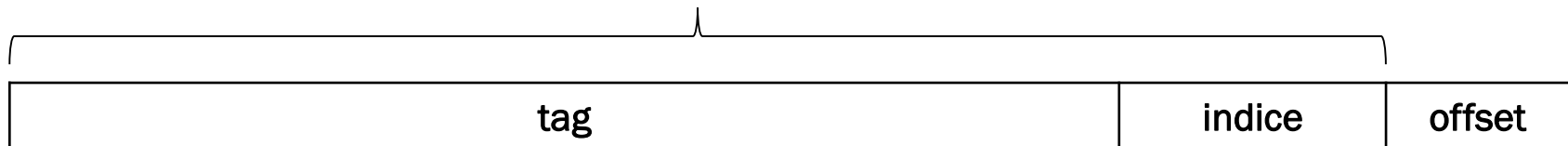
1 bit di **validità** (ed eventualmente i bit **Used** e **Dirty**)

il **tag**: 32 bit – bit di offset – bit di indice di linea (set)

il **blocco**: 8 [bit] × numero di byte = $N \times 32$

I bit necessari per l'**offset** (di blocco e word) sono $\log_2(N \times 4)$ e per l'**indice** $\log_2(S)$

Numero di blocco



Esempio: cache a **4 vie**, con **8 set** e blocchi da **4 word**

dimensione blocco = 4 word × 4 byte = 16 byte = 16 × 8 bit = **128 bit**

numero di bit per **offset** = $\log_2(4) + \log_2(4) = 4$ bit

numero di bit per **indice** = $\log_2(8) = 3$ bit

numero di bit per **tag** = 32 – 3 – 4 = **25 bit**

Dimensione della cache in bit:

4 vie × 8 set × (1 bit valid. + 128 bit blocco + 25 bit tag) = 32 × 154 = **4928 bit**

Politica di rimpiazzo della cache

Come scegliere quale blocco sostituire quando un set della cache è pieno?

Politica di rimpiazzo della cache

- **LRU (Least Recently Used):** sostituire il blocco «**più vecchio**» (**ultimo accesso**)
 - Approssimazione: si associa un bit (**Used**) a ciascuna linea
 - Ad ogni accesso si pone **Used = 1**
 - Lo si azzera allo scadere di un intervallo di tempo
 - Le linee con **Used = 1** sono «**più recenti**»
 - Le linee con **Used = 0** sono «**più vecchie**»
 - Può essere costoso mantenere questa informazione
- **LFU (Least Frequently Used):** sostituire il blocco «**meno usato**»
 - Si può associare un contatore a ciascun set ed aggiornarlo ad ogni accesso
 - Hardware necessario più complicato
- **Random:** sostituire un blocco «**a caso**» (per esempio in sequenza)

Tipi di MISS

Data una cache a **W vie** con **S set**,

un accesso alla cache può causare MISS per tre motivi:

1. **Cold start:** prima richiesta del dato/istruzione
2. **Conflict:** il blocco è stato sovrascritto (solo) per via del grado di associatività della cache
 - in una fully-associative cache, se richiesto dopo meno di $W \times S$ altri blocchi, sarebbe una HIT
3. **Capacity:** il blocco è stato sovrascritto per via della capienza della cache
 - anche in una fully-associative cache, se richiesto dopo più di $W \times S$ altri blocchi, causerebbe comunque una MISS

Quali parametri della cache hanno influenza sui tre tipi di MISS?

1. **Cold start:** la dimensione del blocco ————— **prossimità spaziale**
 - blocchi più grandi contengono più dati/istruzioni → il numero di cold miss diminuisce
2. **Conflict:** il grado di associatività
 - cache con più vie → meno conflitti
3. **Capacity:** la dimensione della cache
 - maggior numero di linee → maggior numero di richieste gestibili

Politiche di scrittura

Come aggiornare la memoria principale (RAM) quando il dato è modificato in cache?

Politiche di scrittura

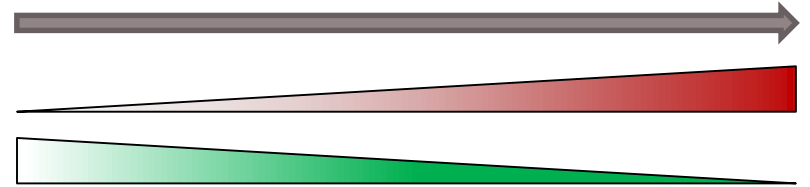
- **Write through:** ad ogni modifica viene aggiornato il blocco in memoria (RAM)
 - **Pro:** in presenza di cache multiple la coerenza dei dati viene mantenuta
 - **Contro:** per la località degli accessi se avvengono più scritture nello stesso blocco si perde molto tempo
- **Write back:** il blocco viene aggiornato solo quando viene sostituito
 - **Pro:** la scrittura del blocco in memoria avviene raramente (è un blocco «vecchio») → cache molto più veloce
 - **Contro:** il contenuto della cache non è più coerente con quello della memoria principale → complicando i sistemi multiprocessore e multicache

Ottimizzazione: con il bit **Dirty** (modificato) si può evitare di scrivere sulla RAM i blocchi non modificati

Influenza dei parametri della cache

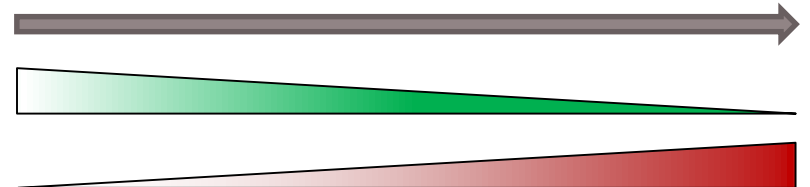
Dimensione del blocco

- ↑ Tempo di MISS (trasferimento da RAM)
- ↓ Percentuale di MISS di tipo **cold start**



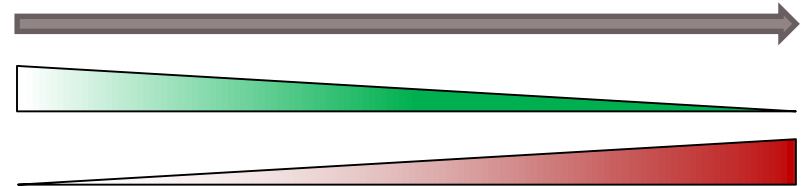
Dimensione della cache (numero di linee)

- ↓ Percentuale di MISS di **capacità**
- ↑ Costo (più memoria)



Associatività (numero di vie)

- ↓ Percentuale di MISS di **conflitto**
- ↑ Costo (più comparatori)



IDEA: migliorare le performance del sistema usando più livelli di cache ottimizzate con parametri diversi (e costi diversi)

Diversa associatività

Sia data la sequenza di accessi a memoria: (0, 8, 0, 6, 8)

Calcolare il numero di HIT e MISS date le cache con

1. direct mapping:

- 4 linee di blocchi da 1 word

2. set-associative mapping:

- 2 vie, 2 set (4 linee) con blocchi da 1 word
- politica di rimpiazzo LRU

3. fully-associative mapping:

- 4 vie, 1 set (4 linee) con blocchi da 1 word
- politica di rimpiazzo LRU

Diversa associatività

Sia data la sequenza di accessi a memoria: (0, 8, 0, 6, 8)

1 via, 4 set

MISS = 100%

Blocchi da 1 word, 4 linee, <u>direct mapped</u>					
#blocco	0	8	0	6	8
index	0	0	0	2	0
tag	0	2	0	1	2
HIT/MISS	MISS (cold start)	MISS (cold start)	MISS	MISS (cold start)	MISS

2 vie, 2 set

MISS = 80%

Blocchi da 1 word, 2 set, <u>2 vie con rimpiazzo LRU</u>					
#blocco	0	8	0	6	8
index	0	0	0	0	0
tag	0	4	0	3	4
HIT/MISS	MISS (cold start)	MISS (cold start)	HIT	MISS (cold start)	MISS

4 vie, 1 set

MISS = 60%

Blocchi da 1 word, 1 set, <u>4 vie con rimpiazzo LRU</u>					
#blocco	0	8	0	6	8
tag	0	8	0	6	8
HIT/MISS	MISS (cold start)	MISS (cold start)	HIT	MISS (cold start)	HIT

Diversa associatività

Sia data la sequenza di accessi a memoria: (0, 8, 0, 6, 8)

1 via, 4 set

MISS = 100%

Blocchi da 1 word, 4 linee, <u>direct mapped</u>					
#blocco	0	8	0	6	8
index	0	0	0	2	0
tag	0	2	0	1	2
HIT/MISS	MISS (cold start)	MISS (cold start)	MISS (conflict)	MISS (cold start)	MISS (conflict)

2 vie, 2 set

MISS = 80%

Blocchi da 1 word, 2 set, <u>2 vie con rimpiazzo LRU</u>					
#blocco	0	8	0	6	8
index	0	0	0	0	0
tag	0	4	0	3	4
HIT/MISS	MISS (cold start)	MISS (cold start)	HIT	MISS (cold start)	MISS (conflict)

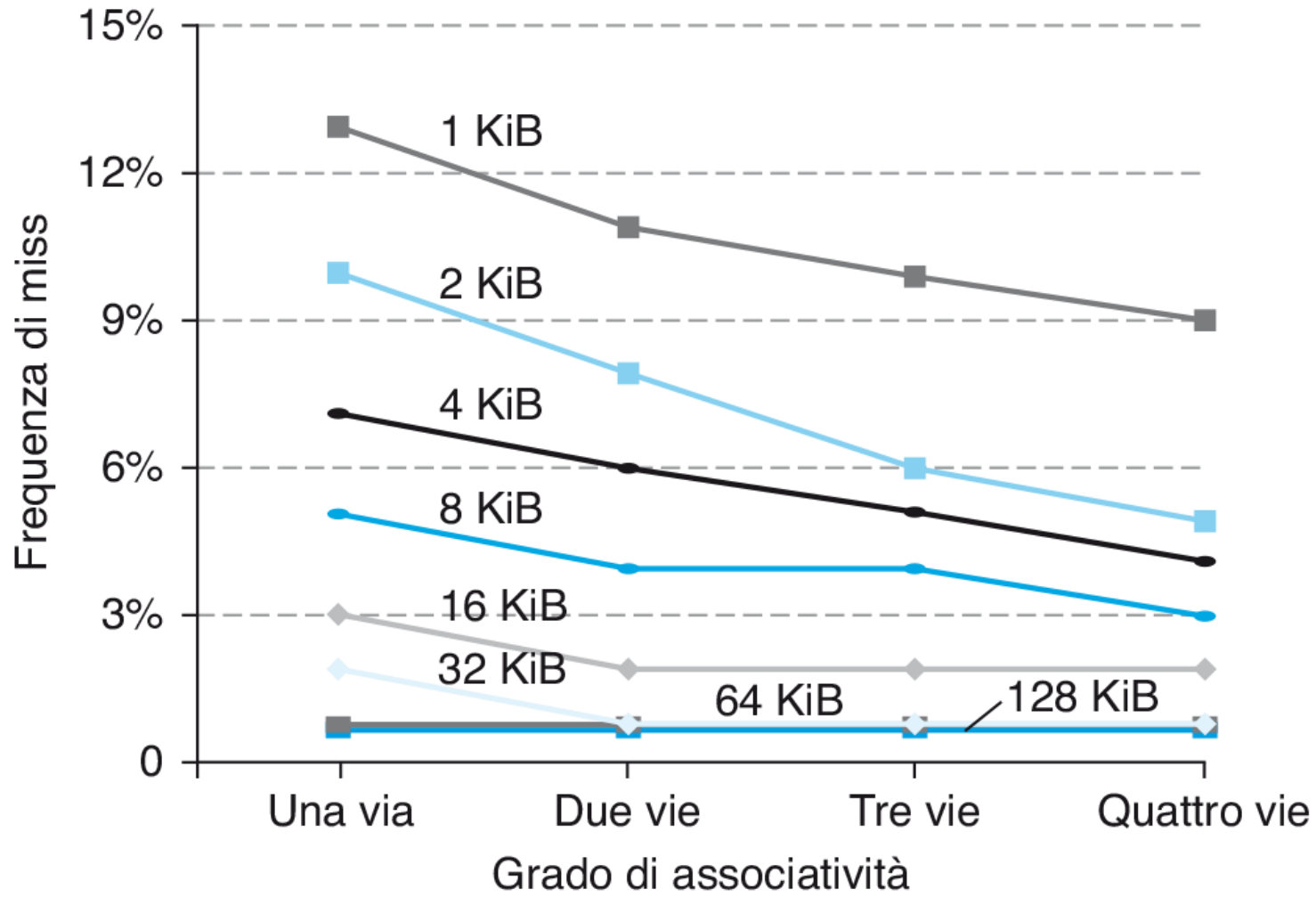
sarebbe
MISS
(capacity)
se fosse
MISS anche
per una fully
associative
cache della
stessa
dimensione

4 vie, 1 set

MISS = 60%

Blocchi da 1 word, 1 set, <u>4 vie con rimpiazzo LRU</u>					
#blocco	0	8	0	6	8
tag	0	8	0	6	8
HIT/MISS	MISS (cold start)	MISS (cold start)	HIT	MISS (cold start)	HIT

Miss rate v. grado di associatività



Cache multi-livello

In una memoria multi-livello:

- ogni **HIT** fornisce i dati a tutti i livelli superiori
- solo le **MISS** fanno richieste al livello subito inferiore
- ogni accesso impiega il tempo del livello più basso che ha risposto

Esempio di calcolo dei tempi se supponiamo che:

- HIT su L1: **2 ns**
- HIT su L2: **30 ns**
- accesso a RAM (ovvero MISS su L2 e HIT su RAM): **100 ns**
- clock della CPU da **2 GHz** (2 CC per ns) e **3 CPI** (CC per istr.)

Dati: perc. di MISS su L1 = 15% → % HIT su L1 = 85%

perc. di MISS su L2 = 20% → % HIT su L2 = 80%

Il **tempo medio** di un **accesso in memoria** sarà

$$\begin{aligned} & \underline{0.85 \times 2 \text{ ns}} + \underline{0.15 \times 0.8 \times 30 \text{ ns}} + \underline{0.15 \times 0.2 \times 100 \text{ ns}} = \\ & = \underline{1.7 \text{ ns}} + \underline{3.6 \text{ ns}} + \underline{3 \text{ ns}} = \underline{8.3 \text{ ns}} \end{aligned}$$

Ovvero $8.3 \times 2 = \mathbf{16.6}$ clock per accesso (1 ogni 5.5 istruzioni)

Con la sola L1: $\underline{0.85 \times 2 \text{ ns}} + \underline{0.15 \times 100 \text{ ns}} = \underline{1.7 \text{ ns}} + \underline{15 \text{ ns}} = \mathbf{16.7 \text{ ns}}$ (2 volte più lento)

